AER 1513 - Assignment 2

Student: Feiyang Wu, ID:1012831205

Introduction

This report follows the assignment's numbering and the lecture notations. We study a differential-drive robot with state:

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}, \quad u_k = \begin{bmatrix} v_k \\ \omega_k \end{bmatrix},$$

Where sampling period $T \approx 0.1$ s, and a range–bearing sensor located at a forward offset d from the geometric center of the robot. The position of a set of fixed landmarks (x^l, y^l) is given. The EKF fuses odometry (process model) with range–bearing measurements (observation model). We adopt the noise statistics provided in dataset and do not recompute them; all plots referenced are those given in the assignment handout.

Question 1 — Noise Assumptions & Covariances

1.1 Problem restatement

Assume zero-mean Gaussian noises for process and measurement. Using the noise statistics provided in dataset, (i) state whether the Gaussian assumptions are reasonable and (ii) specify the measurement covariance R and the process covariance Q_k used in the EKF.

1.2 Notation (consistent with the slides)

- State: $\mathbf{x}_k = [x_k \quad y_k \quad \theta_k]^\top$.
- Inputs (odometry): $u_k = [v_k \quad \omega_k]^\top$ (linear and angular speeds).
- Sampling period: T. Sensor offset: d (robot center → sensor).
- Landmark l: (x^l, y^l).
- Process noise: $w_k \sim \mathcal{N}(0, Q_k)$.
- Measurement noise (for landmark l): $n_k^l \sim \mathcal{N}(0, R_k)$.
- Dataset variances: $\sigma_v^2[\frac{m^2}{s^2}], \sigma_\omega^2[\frac{rad^2}{s^2}], \sigma_r^2[m^2], \sigma_\phi^2[rad^2]$.
- Unit: $r[m], \phi[rad], v\left[\frac{m}{s}\right], \omega\left[\frac{rad}{s}\right]$; The sensor offset $d[m]$ is along the body $+x$ diraction

1.3 Motion & observation models

Motion model (unicycle, forward Euler):

$$\begin{aligned} x_k &= x_{k-1} + Tv_k\cos\theta_{k-1}, \\ y_k &= y_{k-1} + Tv_k\sin\theta_{k-1}, \\ \theta_k &= \theta_{k-1} + T\omega_k. \end{aligned}$$

Observation (range–bearing with forward offset d) for a visible landmark l:

$$\Delta x = x^l - x_k - d\cos\theta_k, \quad \Delta y = y^l - y_k - d\sin\theta_k,$$
$$r_k^l = \sqrt{\Delta x^2 + \Delta y^2} + n_{r,k}^l = \sqrt{(x_\ell - x_k - d\cos\theta_k)^2 + (y_\ell - y_k - d\sin\theta_k)^2} + n_{r,k}^l,$$
$$\phi_k^l = \operatorname{atan2}(\Delta y, \Delta x) - \theta_k + n_{b,k}^l = \operatorname{atan2}(y_\ell - y_k - d\sin\theta_k, x_\ell - x_k - d\cos\theta_k) - \theta_k + n_{b,k}^l.$$

(Only use measurements with $r_k^l > 0$; if a distance gate is used, also require $r_k^l \le r_{\max}$.)

Angle normalization: wrap bearing residuals to $(-\pi, \pi]$ when forming the innovation and wrap the updated $\hat{\theta}_k$ to $(-\pi, \pi]$ after correction.

Convention: Jacobians in question 2 are evaluated at $(\hat{x}_{k-1}, v_k, \omega_k)$; that is, inputs use the current sample k, while the linearization for orientation uses $\hat{\theta}_{k-1}$.

1.4 Measurement covariance R (use dataset values)

We model range and bear errors as independent Gaussians:

$$R^l = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix} = \begin{bmatrix} 0.00090036 & 0 \\ 0 & 0.00067143 \end{bmatrix},$$

where σ_r^2 is in m^2 and σ_ϕ^2 in rad^2.

If multiple landmarks are stacked at time k, For each time step k, after selecting valid landmarks:

$$\mathcal{L}_k = \{\ell : r_k^\ell > 0 \text{ and } r_k^\ell \le r_{\max}\},$$

stack the measurement vector is

$$z_k = \left[r_k^{\ell_1}, \phi_k^{\ell_1}, \dots, r_k^{\ell_{M_k}}, \phi_k^{\ell_{M_k}}\right]^\top \in \mathbb{R}^{2M_k},$$

where $M_k = |\mathcal{L}_k|$.

Assuming independence across landmarks and between range and bearing channels, the stacked covariance is block-diagonal:

$$R_k = \operatorname{blkdiag}\Bigg(\underbrace{\operatorname{diag}(\sigma_r^2, \sigma_\phi^2), \dots, \operatorname{diag}(\sigma_r^2, \sigma_\phi^2)}_{M_k \text{ blocks}}\Bigg).$$

1.5 Process covariance Q_k (map odometry variances into the state)

Treating odometry uncertainty as input noise with

$$Q_u = \mathrm{diag}(\sigma_v^2, \sigma_\omega^2).$$

The input–state Jacobian at step k is

$$G_k = T \begin{bmatrix} \cos\hat{\theta}_{k-1} & 0 \\ \sin\hat{\theta}_{k-1} & 0 \\ 0 & 1 \end{bmatrix}.$$

Then propagate to state space:

$$Q_k = G_k \Sigma_u G_k^\top = \begin{bmatrix} T^2\cos^2\theta\, v_{\text{var}} & T^2\cos\theta\sin\theta\, v_{\text{var}} & 0 \\ T^2\cos\theta\sin\theta\, v_{\text{var}} & T^2\sin^2\theta\, v_{\text{var}} & 0 \\ 0 & 0 & T^2\omega_{\text{var}} \end{bmatrix}, \quad \theta = \hat{\theta}_{k-1}.$$

$$Q_k = \begin{bmatrix} 4.18769326e-05 & 9.86862788e-06 & 0 \\ 9.86862788e-06 & 2.32561962e-06 & 0 \\ 0 & 0 & 8.18608753e-05 \end{bmatrix}$$

Interpretation: Forward-speed noise projects onto both x and y, while heading noise affects only θ in this first-order discretization.

1.6 Are the Gaussian, zero-mean assumptions reasonable?

Yes. The assignment handout (based on motion-capture ground truth) shows histograms for both sensor (range/bearing) in figure 2.5 and odometry (linear/angular speed) errors in figure 2.6 that are (i) unimodal and symmetric and (ii) centered near zero. This supports modeling n_k^l and w_k as zero-mean Gaussians. Since we use the dataset-provided variances directly (rather than tuning), our Q_k and R reflect measured sensor/odometry uncertainty.

Question 2 — EKF Jacobians

2.1 Problem restatement

Derive the Jacobians required by the EKF for the motion model and the range–bearing observation model with a forward sensor offset distance d. Provide expressions of all the Jacobians.

2.2 Notation (aligned with slides)

- State: $\mathbf{x}_k = [x_k, \quad y_k, \quad \theta_k]^\top$.
- Inputs (odometry): $u_k = [v_k, \quad \omega_k]^\top$ (linear, angular speed).
- Sampling period: T.

- Sensor offset (forward along body x-axis): d.
- Landmark l: (x^l, y^l).
- Motion function: $\mathbf{x}_k = f(\mathbf{x}_{k-1}, u_k)$.
- Measurement function (for landmark l): $\mathbf{z}_k^l = g(\mathbf{x}_k) + n_k^l$, with $\mathbf{z}_k^l = [r_k^l, \quad \phi_k^l]^\top$.

2.3 Motion model & Jacobians

Motion model (forward Euler):

$$\begin{aligned} x_k &= x_{k-1} + T v_k \cos\theta_{k-1}, \\ y_k &= y_{k-1} + T v_k \sin\theta_{k-1}, \\ \theta_k &= \theta_{k-1} + T\omega_k. \end{aligned}$$

State Jacobian $F_k = \partial f / \partial \mathbf{x}_{k-1} \in \mathbb{R}^{3\times 3}$:

$$F_k = \begin{bmatrix} 1 & 0 & -T v_k \sin\theta_{k-1} \\ 0 & 1 & T v_k \cos\theta_{k-1} \\ 0 & 0 & 1 \end{bmatrix}$$

Input-to-state Jacobian $G_{k-1} = \partial f / \partial u_k \in \mathbb{R}^{3\times 2}$:

$$G_{k-1} = \begin{bmatrix} T\cos\theta_{k-1} & 0 \\ T\sin\theta_{k-1} & 0 \\ 0 & T \end{bmatrix}$$

Use G_{k-1} to map odometry noise into the state for the process covariance:

$$Q_k = G_{k-1}\, \mathrm{diag}(\sigma_v^2, \sigma_\omega^2)\, G_{k-1}^\top.$$

2.4 Observation model (with offset d) & Jacobian

The sensor position in the world at time k is

$$x_s = x_k + d\cos\theta_k, \quad y_s = y_k + d\sin\theta_k.$$

Define relative geometry to landmark l:

$$\Delta x = x^l - x_s, \quad \Delta y = y^l - y_s, \quad q = \Delta x^2 + \Delta y^2, \quad r = \sqrt{q}.$$

Observation function:

$$g(\mathbf{x}_k) = \begin{bmatrix} r \\ \phi \end{bmatrix} = \begin{bmatrix} \sqrt{(x^l - x_k - d\cos\theta_k)^2 + (y^l - y_k - d\sin\theta_k)^2} \\ \mathrm{atan2}(\Delta y, \Delta x) - \theta_k \end{bmatrix}.$$

Observation Jacobian $H_k^l = \partial g / \partial \mathbf{x}_k \in \mathbb{R}^{2\times 3}$:

$$H_k^l = \begin{bmatrix} -\frac{\Delta x}{r} & -\frac{\Delta y}{r} & \frac{d}{r}(\Delta x \sin\theta_k - \Delta y \cos\theta_k) \\ \frac{\Delta y}{q} & -\frac{\Delta x}{q} & -\frac{d}{q}(\Delta y \sin\theta_k + \Delta x \cos\theta_k) - 1 \end{bmatrix}$$

Noise Jacobian (observation noise to measurement space) is the identity, $V_k^l = I_{2\times 2}$.

Notes.

- Bearing residuals must be wrapped to $(-\pi, \pi]$ both when forming the innovation and when normalizing angles in the state after updates.
- If there are multiple landmarks at the same timestamp, stack rows of H_k^l to build H_k, and use $R_k = \text{blkdiag}(R, \dots, R)$.

2.5 EKF step (where the Jacobians plug in)

Predictor (time update)

$$\boxed{\breve{\mathbf{P}}_k = \mathbf{F}_{k-1}\widehat{\mathbf{P}}_{k-1}\mathbf{F}_{k-1}^\top + \mathbf{Q}_k'}$$

$$\boxed{\breve{\mathbf{x}}_k = f(\hat{\mathbf{x}}_{k-1}, \mathbf{v}_k, 0)}$$

- $\mathbf{F}_{k-1} = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{(\hat{\mathbf{x}}_{k-1}, \mathbf{v}_k)}$ is the motion Jacobian.
- $\mathbf{Q}_k'$ is the process covariance in state space. In our setup with additive odometry noise, $\mathbf{Q}_k' = \mathbf{L}_k \mathbf{Q}_u \mathbf{L}_k^\top$, and specifically $\mathbf{Q}_k' = \mathbf{G}_{k-1}^{(u)} \mathbf{Q}_u \left(\mathbf{G}_{k-1}^{(u)}\right)^\top$, where $\mathbf{G}_{k-1}^{(u)} = \frac{\partial f}{\partial \mathbf{v}}$ and $\mathbf{Q}_u = \text{diag}(v_{\text{var}}, \omega_{\text{var}})$.

Kalman gain

$$\boxed{\mathbf{K}_k = \breve{\mathbf{P}}_k \mathbf{G}_k^\top \left(\mathbf{G}_k \breve{\mathbf{P}}_k \mathbf{G}_k^\top + \mathbf{R}_k'\right)^{-1}}$$

- $\mathbf{G}_k = \left.\frac{\partial g}{\partial \mathbf{x}}\right|_{\breve{\mathbf{x}}_k}$ is the observation Jacobian (slides use $\mathbf{G}$ where many texts use H).
- $\mathbf{R}_k' = \mathbf{V}_k \mathbf{R} \mathbf{V}_k^\top$ is the observation covariance in measurement space; with additive range–bearing noise we have $\mathbf{V}_k = \mathbf{I}$ and $\mathbf{R}_k' = \mathbf{R}$.

Corrector (measurement update)

$$\boxed{\widehat{\mathbf{P}}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{G}_k)\breve{\mathbf{P}}_k}$$

$$\boxed{\hat{\mathbf{x}}_k = \breve{\mathbf{x}}_k + \mathbf{K}_k(\mathbf{y}_k - g(\breve{\mathbf{x}}_k, 0))}$$

Question 3 — EKF with a variable Number of Exteroceptive Measurements

3.1 Problem statement

At each time step k, the laser scanner observes a variable amount of the L available landmarks (here $L = 17$). Denote the visible set by $\mathcal{M}_k \subseteq \{1, \dots, L\}$ with size $m_k = |\mathcal{M}_k|$. We must write EKF equations that remain valid for any m_k (including $m_k = 0$). The motion model and the observation model are the same as in previous questions, only the *measurement update* is modified to accommodate a changing number of observations.

3.2 Models and notation

- State $\mathbf{x}_k = [x_k, \quad y_k, \quad \theta_k]^\top \in \mathbb{R}^3$.
- Input (odometry) $\mathbf{v}_k = [v_k, \quad \omega_k]^\top$.
- Motion function $\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{v}_k, 0)$ with process noise $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_k)$.
- Measurement function for landmark l: $\mathbf{y}_k^l = g(\mathbf{x}_k, 0) + \mathbf{n}_k^l$ (range–bearing, with sensor forward offset d; $\mathbf{n}_k^l \sim \mathcal{N}(0, \mathbf{R})$).
- Linearization:

$$\mathbf{F}_{k-1} = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{(\hat{\mathbf{x}}_{k-1}, \mathbf{v}_k)}, \quad \mathbf{G}_k^l = \left.\frac{\partial g}{\partial \mathbf{x}}\right|_{\check{\mathbf{x}}_k} \text{ (per-landmark).}$$

- Covariance mappings:
  - Process in state space: $\mathbf{Q}_k' = \mathbf{L}_k \mathbf{Q} \mathbf{L}_k^\top$ (with additive odometry noise often $\mathbf{L}_k = \frac{\partial f}{\partial \mathbf{v}}$, so $\mathbf{Q}_k' = \mathbf{G}_{k-1}^{(u)} \mathbf{Q}_u \left(\mathbf{G}_{k-1}^{(u)}\right)^\top$).
  - Measurement in measurement space: $\mathbf{R}_k' = \mathbf{V}_k \mathbf{R} \mathbf{V}_k^\top$. For additive range–bearing noise, $\mathbf{V}_k = \mathbf{I} \Longrightarrow \mathbf{R}_k' = \mathbf{R}$.

3.3 Predictor (unchanged)

$$\boxed{\check{\mathbf{P}}_k = \mathbf{F}_{k-1} \hat{\mathbf{P}}_{k-1} \mathbf{F}_{k-1}^\top + \mathbf{Q}_k'}$$

$$\boxed{\check{\mathbf{x}}_k = f(\hat{\mathbf{x}}_{k-1}, \mathbf{v}_k, 0)}$$

3.4 Corrector with a variable number of measurements (stacked form)

For each visible landmark $l \in \mathcal{M}_k$ (two channels $[r, \phi]$):

- Predicted measurement $\check{\mathbf{y}}_k^l = g(\check{\mathbf{x}}_k, 0) \in \mathbb{R}^2$.

- Innovation $\boldsymbol{\eta}_k^l = \mathbf{y}_k^l - \check{\mathbf{y}}_k^l$, with the *bearing component wrapped to* $(-\pi, \pi]$.
- Jacobian $\mathbf{G}_k^l = \left.\frac{\partial g}{\partial \mathbf{x}}\right|_{\check{\mathbf{x}}_k} \in \mathbb{R}^{2\times 3}$.

Stack all visible landmarks (row-wise):

$$\mathbf{G}_k = \begin{bmatrix} \mathbf{G}_k^{l_1} \\ \vdots \\ \mathbf{G}_k^{l_{m_k}} \end{bmatrix} \in \mathbb{R}^{2m_k\times 3}, \quad \boldsymbol{\eta}_k = \begin{bmatrix} \boldsymbol{\eta}_k^{l_1} \\ \vdots \\ \boldsymbol{\eta}_k^{l_{m_k}} \end{bmatrix} \in \mathbb{R}^{2m_k}.$$

Assuming independent range–bearing noises, build a *block-diagonal* measurement covariance:

$$\mathbf{R}_k' = \underbrace{\mathrm{blkdiag}(\mathbf{R}, \dots, \mathbf{R})}_{m_k \text{ blocks}} \in \mathbb{R}^{2m_k\times 2m_k}.$$

Then the EKF update retains the *lecture form* (only dimensions change with m_k):

$$\boxed{\mathbf{K}_k = \check{\mathbf{P}}_k \mathbf{G}_k^\top \left(\mathbf{G}_k \check{\mathbf{P}}_k \mathbf{G}_k^\top + \mathbf{R}_k'\right)^{-1}}$$

$$\boxed{\hat{\mathbf{P}}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{G}_k)\check{\mathbf{P}}_k}$$

$$\boxed{\hat{\mathbf{x}}_k = \check{\mathbf{x}}_k + \mathbf{K}_k \boldsymbol{\eta}_k} \quad \text{wrap the heading in } \hat{\mathbf{x}}_k.$$

Dimensions (state $n = 3$): $\mathbf{K}_k \in \mathbb{R}^{3\times 2m_k}$, $\mathbf{G}_k \in \mathbb{R}^{2m_k\times 3}$, $\mathbf{R}_k' \in \mathbb{R}^{2m_k\times 2m_k}$.
Edge case $m_k = 0$: skip the update, i.e., $\hat{\mathbf{x}}_k = \check{\mathbf{x}}_k$, $\hat{\mathbf{P}}_k = \check{\mathbf{P}}_k$.
Implementation:

In code, we also try the *Joseph form* for corrector and compare the result

$$\hat{\mathbf{P}}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{G}_k)\check{\mathbf{P}}_k (\mathbf{I} - \mathbf{K}_k \mathbf{G}_k)^\top + \mathbf{K}_k \mathbf{R}_k' \mathbf{K}_k^\top$$

to preserve symmetry and positive semidefiniteness.

Question 4 — EKF Experiments

4.1 Experiment Setup

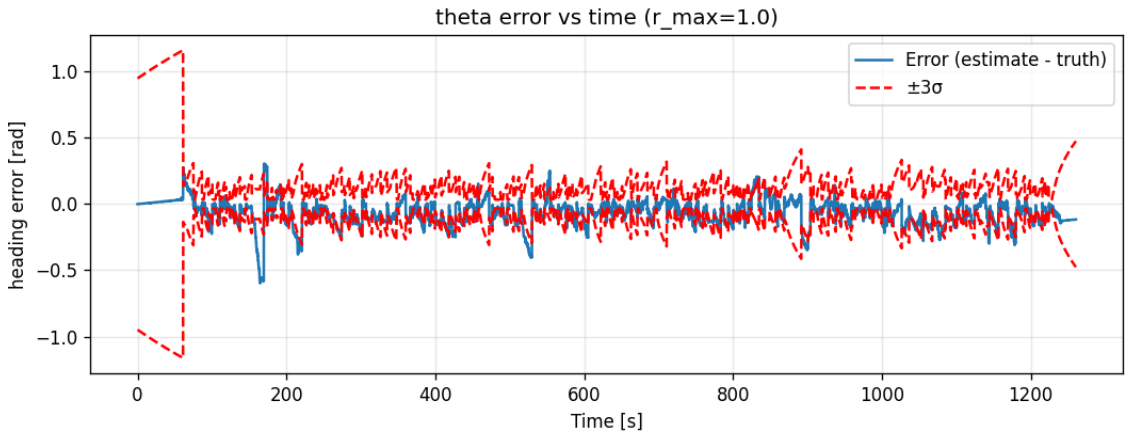
Unicycle model, range–bearing sensing with forward offset d. Gating: use landmarks with $0 < r \le r_{\max}$, $r_{\max} \in \{5,3,1\}$. Bearing residuals and $\hat{\theta}$ are wrapped to $(-\pi, \pi]$. The non-Joseph covariance update $P_k = (I - K_k G_k)P_k^-$ are used for the main run. Jacobians are evaluated:

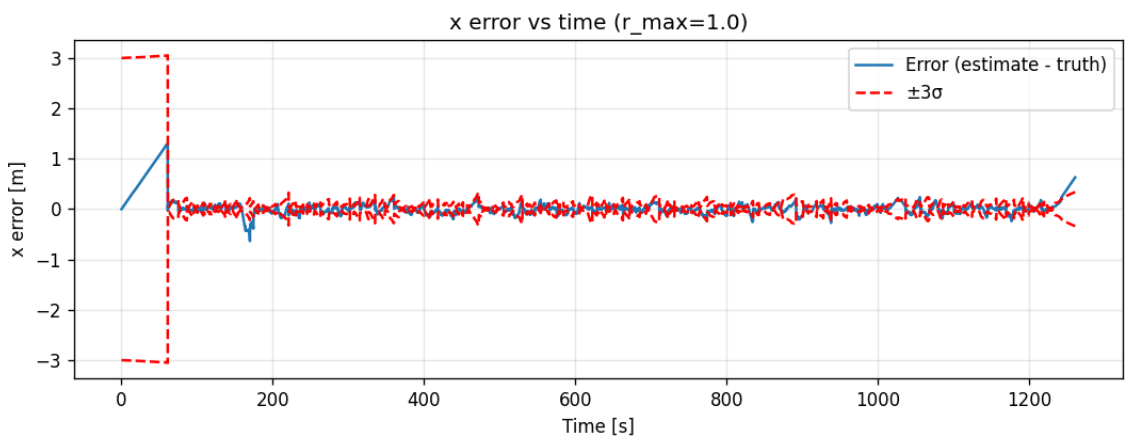
- (a) at the **current estimate** $\hat{x}$ (standard EKF),

- (b) as in (a) but with **poor** $\hat{x}_0$,
- (c) at the **true state** (CRLB-style linearization) while still forming the innovation with $\hat{z} = h(\hat{x}^-)$.

4.2 Q4(a): Good initial mean

Result





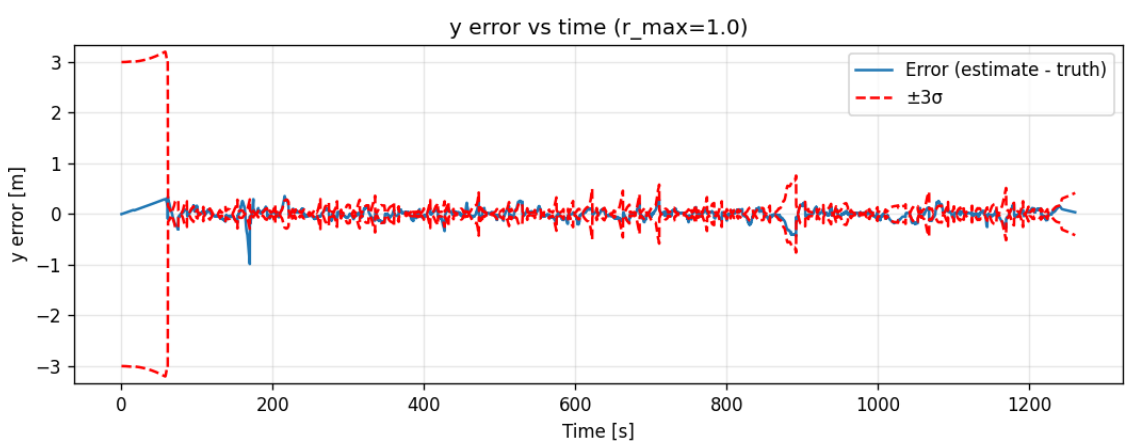


Figure1: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 1m$ with initial condition x_0

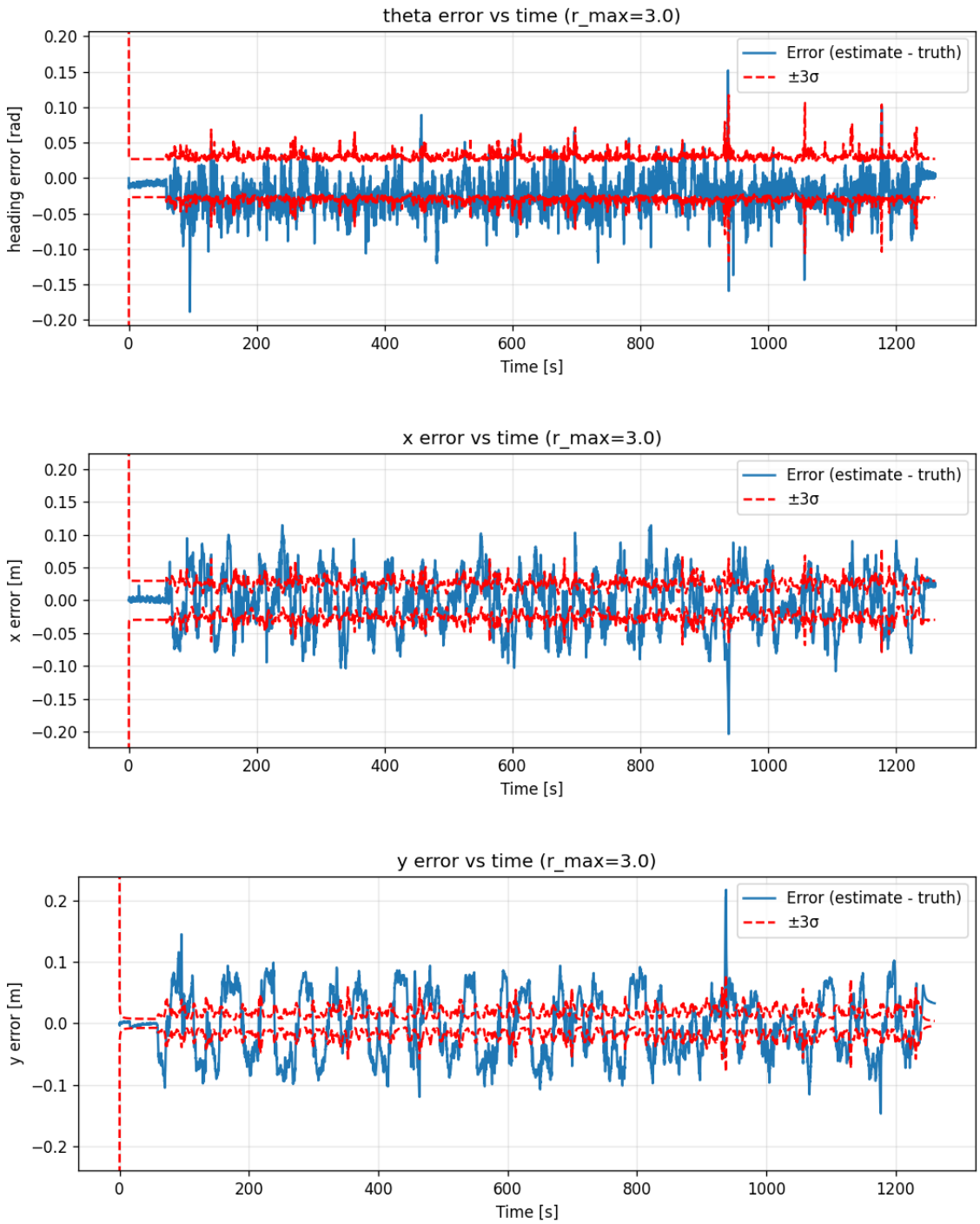


Figure2: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 3m$ with initial condition x_0

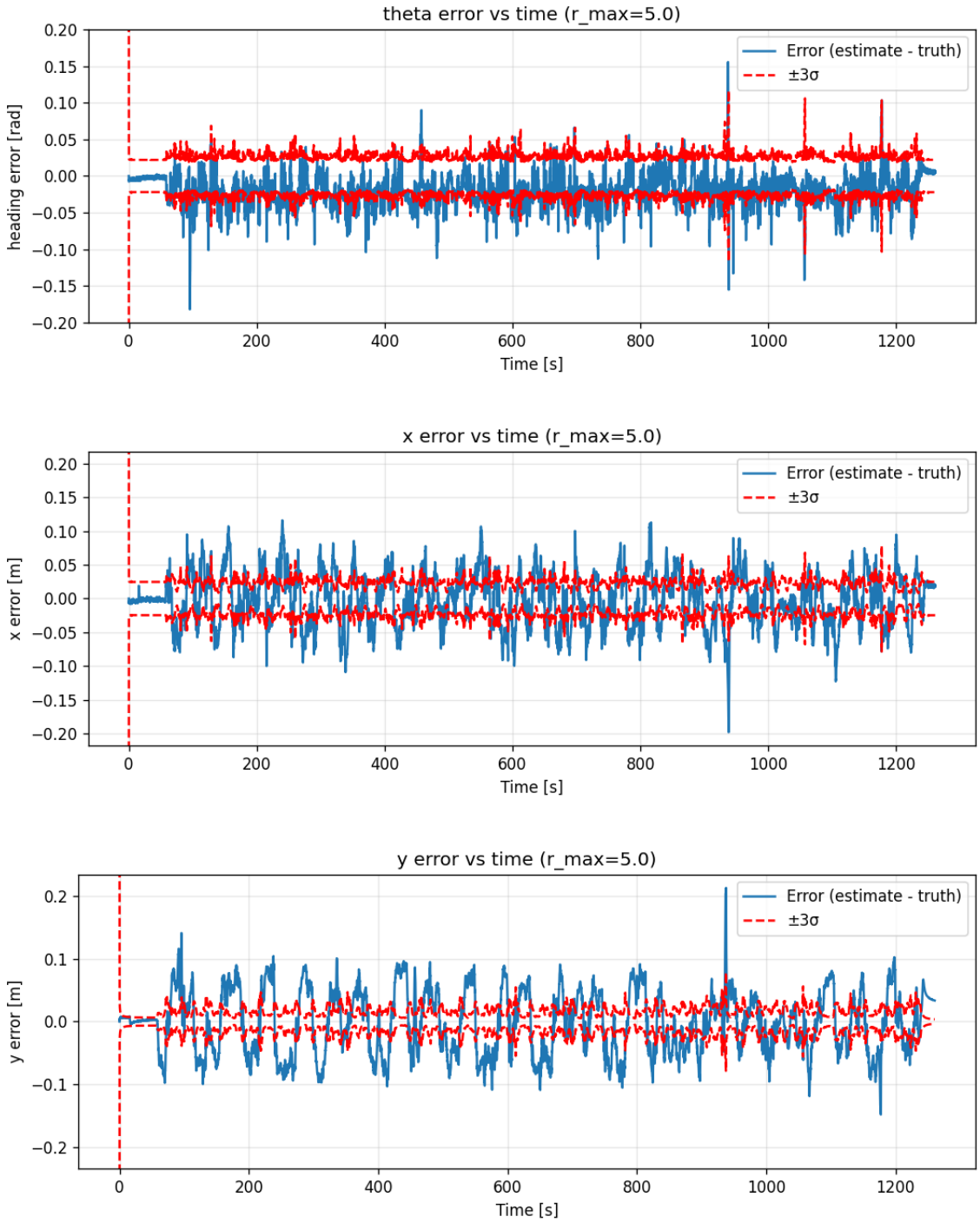


Figure3: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 5m$ with initial condition x_0

Observation

For $r_{\max} = 1$, the $\pm 3\sigma$ bands are wide, and contraction is slower compared to other cases and overall errors are larger.

For $r_{\max} = 3$ and 5, the errors are in the same level while error magnitude of both cases are smaller than error for $r_{\max} = 1$ and the bands are tighter—but the blue error curve still exceeds the red $\pm 3\sigma$ envelope at times (short spikes and occasional brief segments).

Why $\pm 3\sigma$ does not fully bound the error here (Possible reason)

1. **Nonlinearity & linearization error.** The $\pm 3\sigma$ bands came from $\mathrm{diag}(P_k)$ which is computed with Jacobians of a local linear model. When the trajectory turns sharply or the information of robot–landmark geometry is poor, this linear approximation underestimates the actual dispersion, so the error might cross the envelope.

2. **Gating-induced, time-varying information.** The number of landmarks used change with $r_{\max}$ and robot poses. P_k assumes that the linearized measurement set at that instant and drops or additions cause abrupt information changes that the filter probably only partially captured in one step.

3. **Angle wrapping & mixed units.** Heading residuals involve wrap and a highly nonlinear atan2 so near $\pm\pi$ the Gaussian assumption in the linearized bearing becomes weak.

4. **Noise tuning mismatch.** Any mismatch between the true data noise and the chosen Q, R shifts the effective coverage of $\pm 3\sigma$ away from the nominal 99.7%.

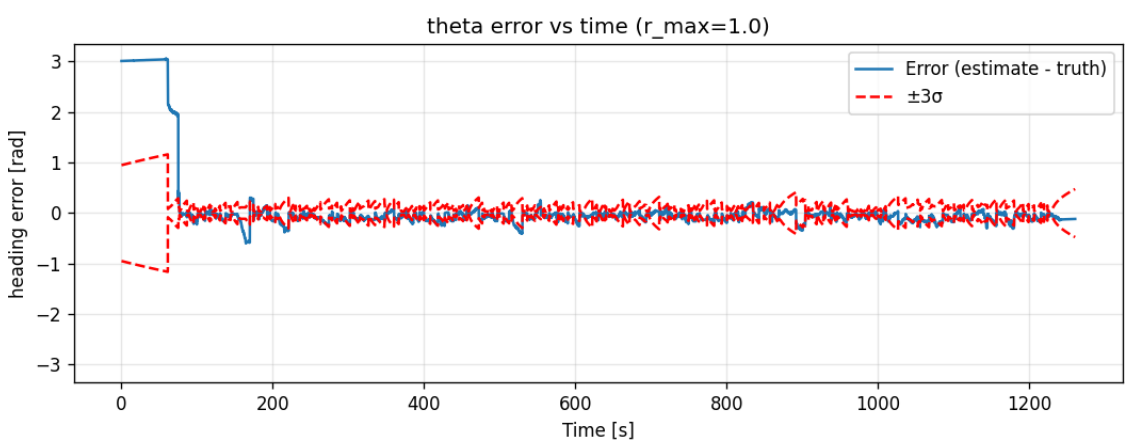
Effect of $r_{\max}$

The result shows the result of $r_{\max} = 3$ and $r_{\max} = 5$ is similar, while 1 is clearly worse.

If $r_{\max}$ greater than 3 m then the extra landmarks only add little information because Jacobian sensitivity decays with range and the geometry is already informative—so information saturates.

4.3 Q4(b): Poor initial mean

Result



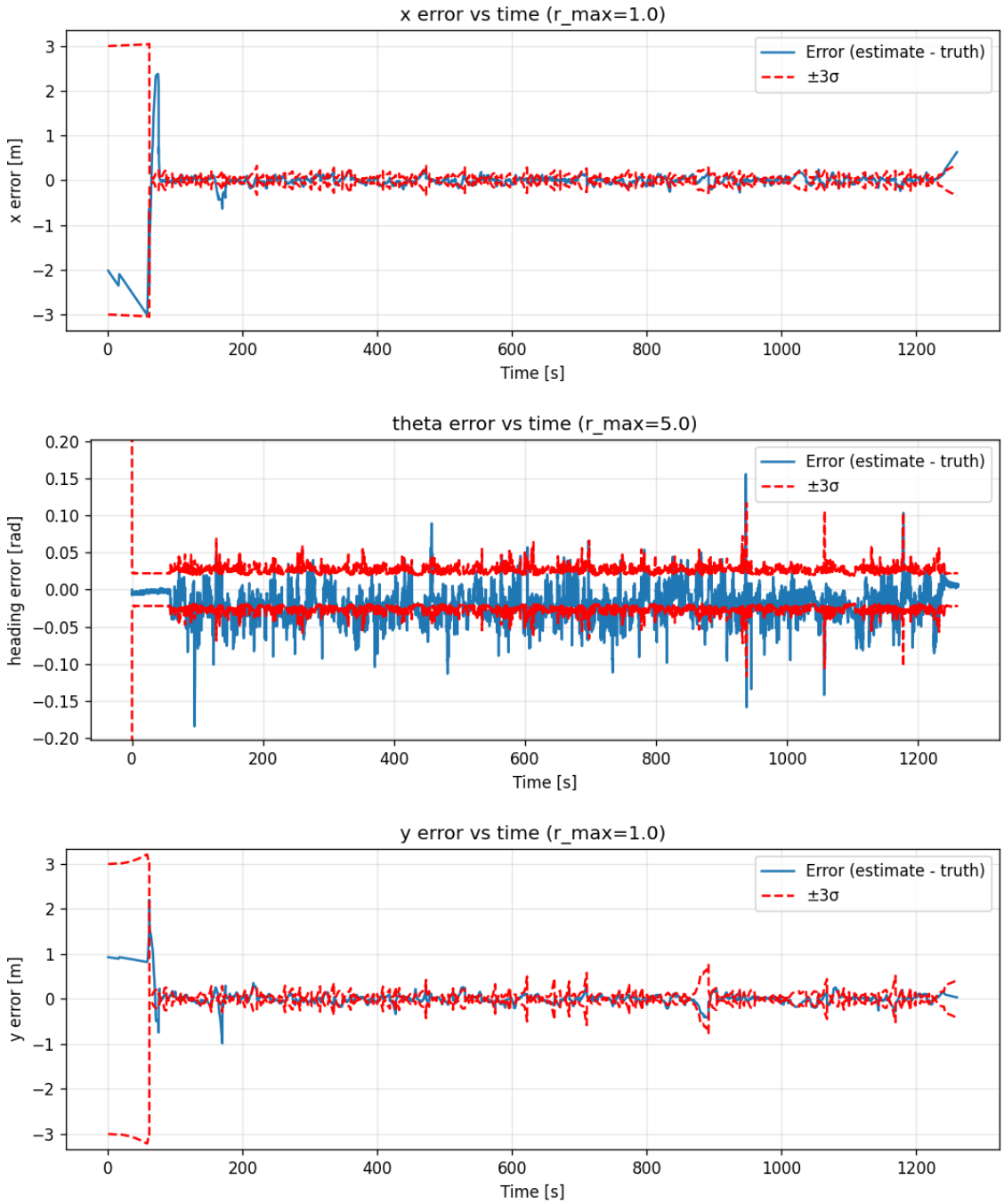


Figure4: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 1m$ with poor initial condition

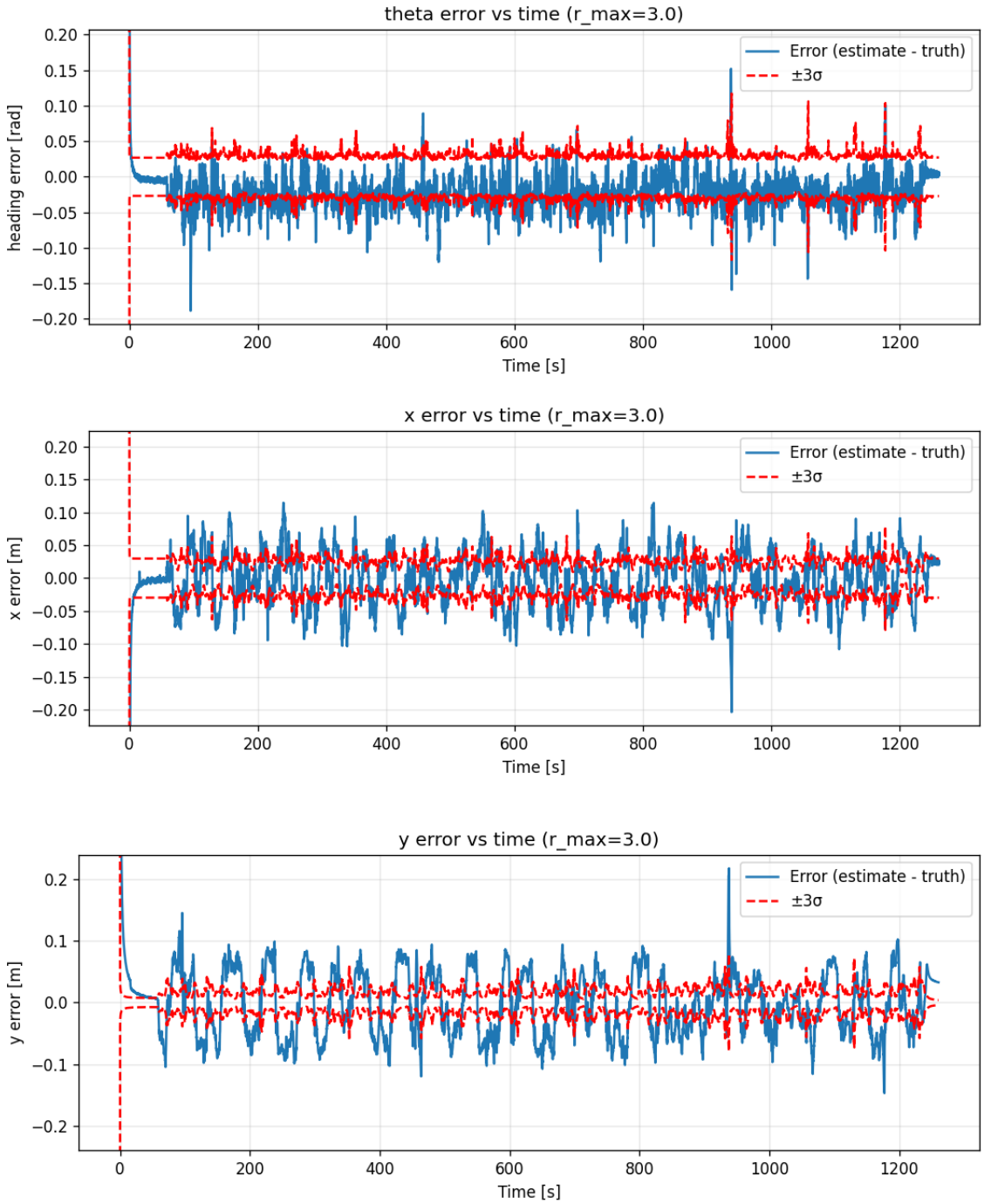


Figure5: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 3m$ with poor initial condition

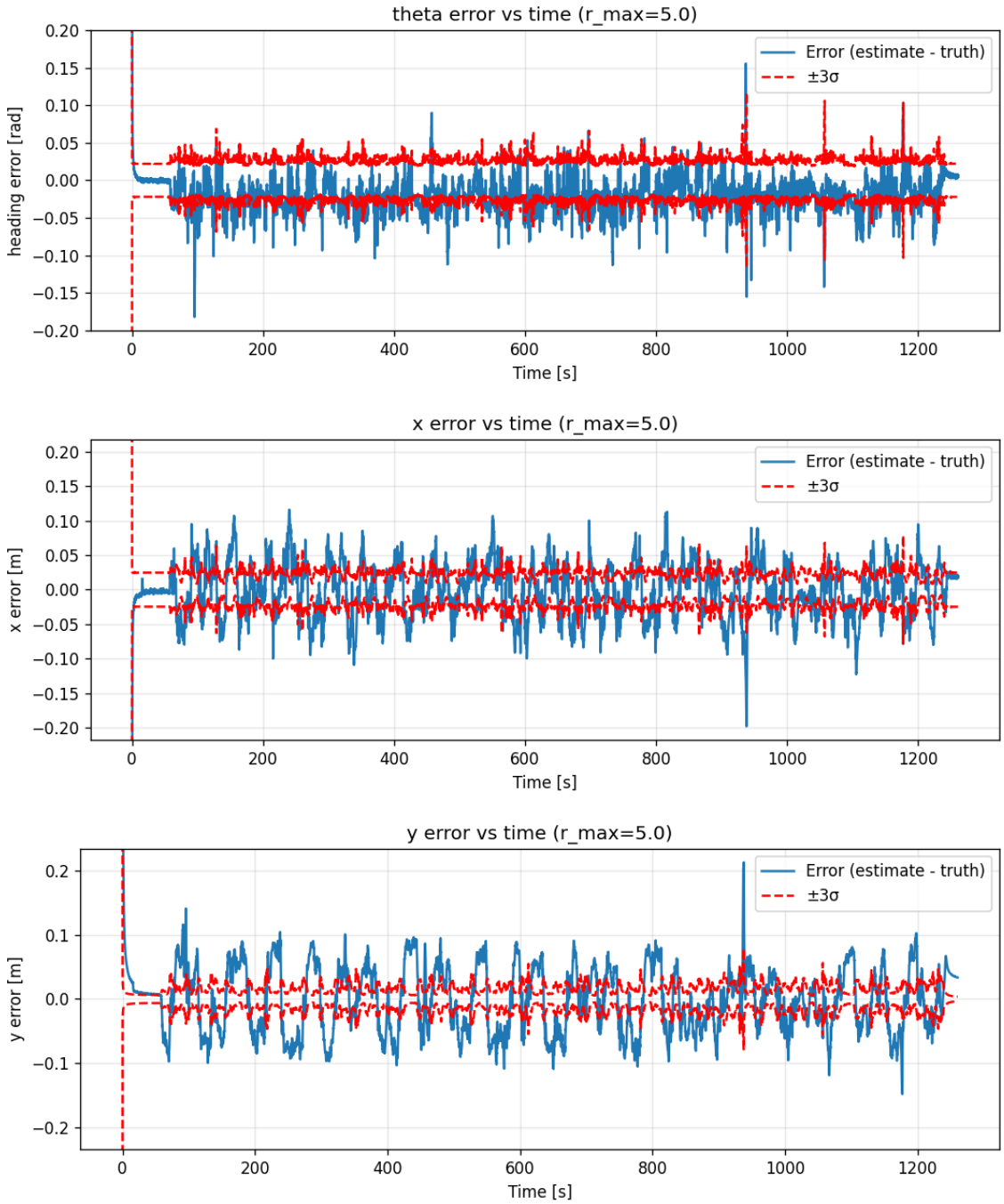


Figure6: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 5m$ with initial condition x_0

Observation

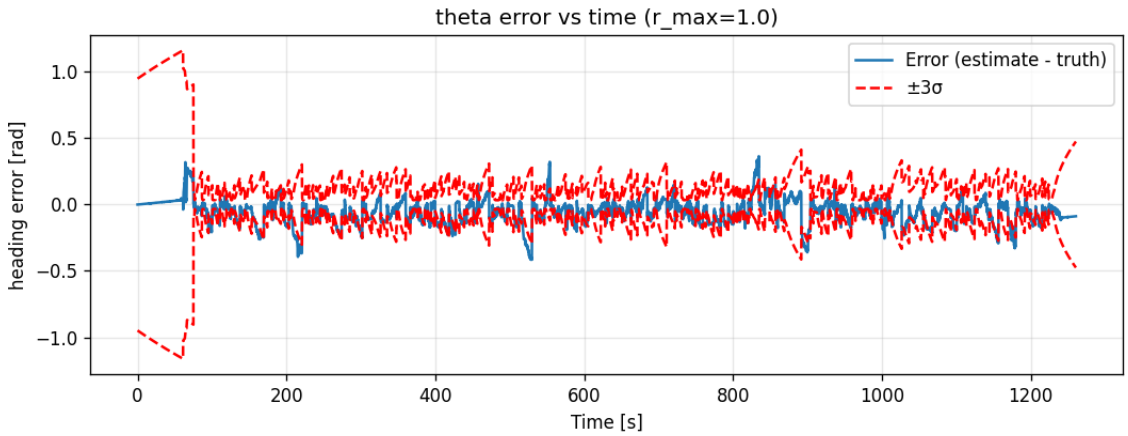
With $\hat{x}_0 = (1,1,0.1)$ the filter exhibits a short transient and then converges to the same steady-state level as (a). The $\pm 3\sigma$ bands again do not strictly contain the error everywhere which is caused by the same reasons as previous question.

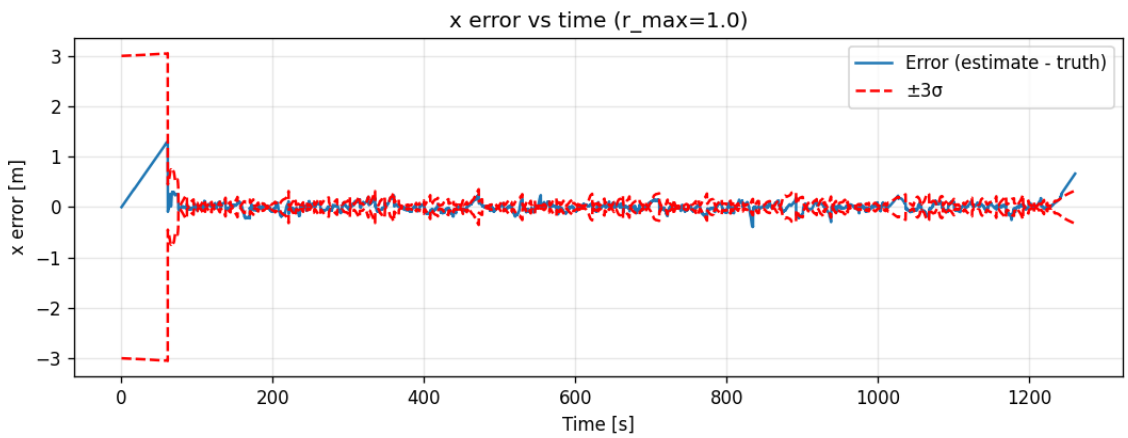
Takeaway

No special retuning for the poor initial condition is required to demonstrate the point: the EKF is robust to the tested bias with the ranking $1 \ll 3 \approx 5$, same as the previous one.

4.4 Q4(c): “CRLB-style” (Jacobians at the truth)

Result





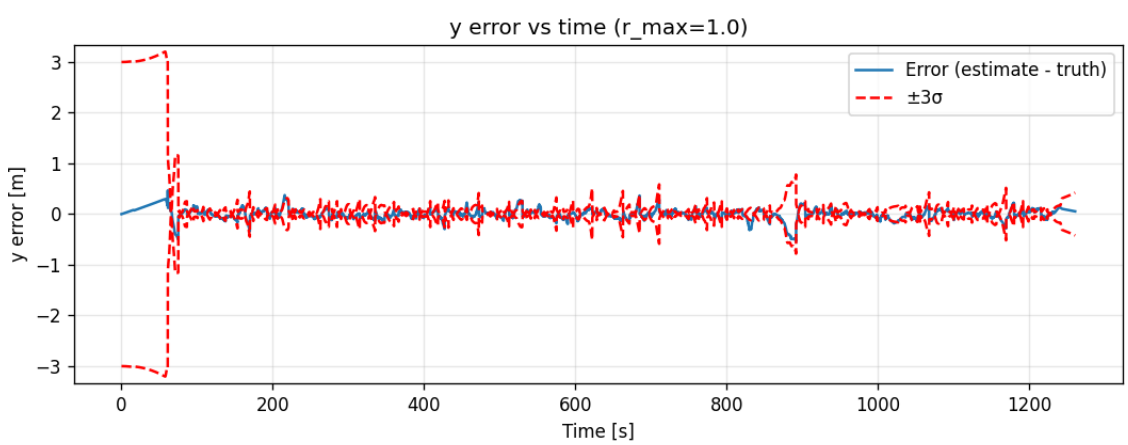


Figure7: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 1m$ with CRLB method

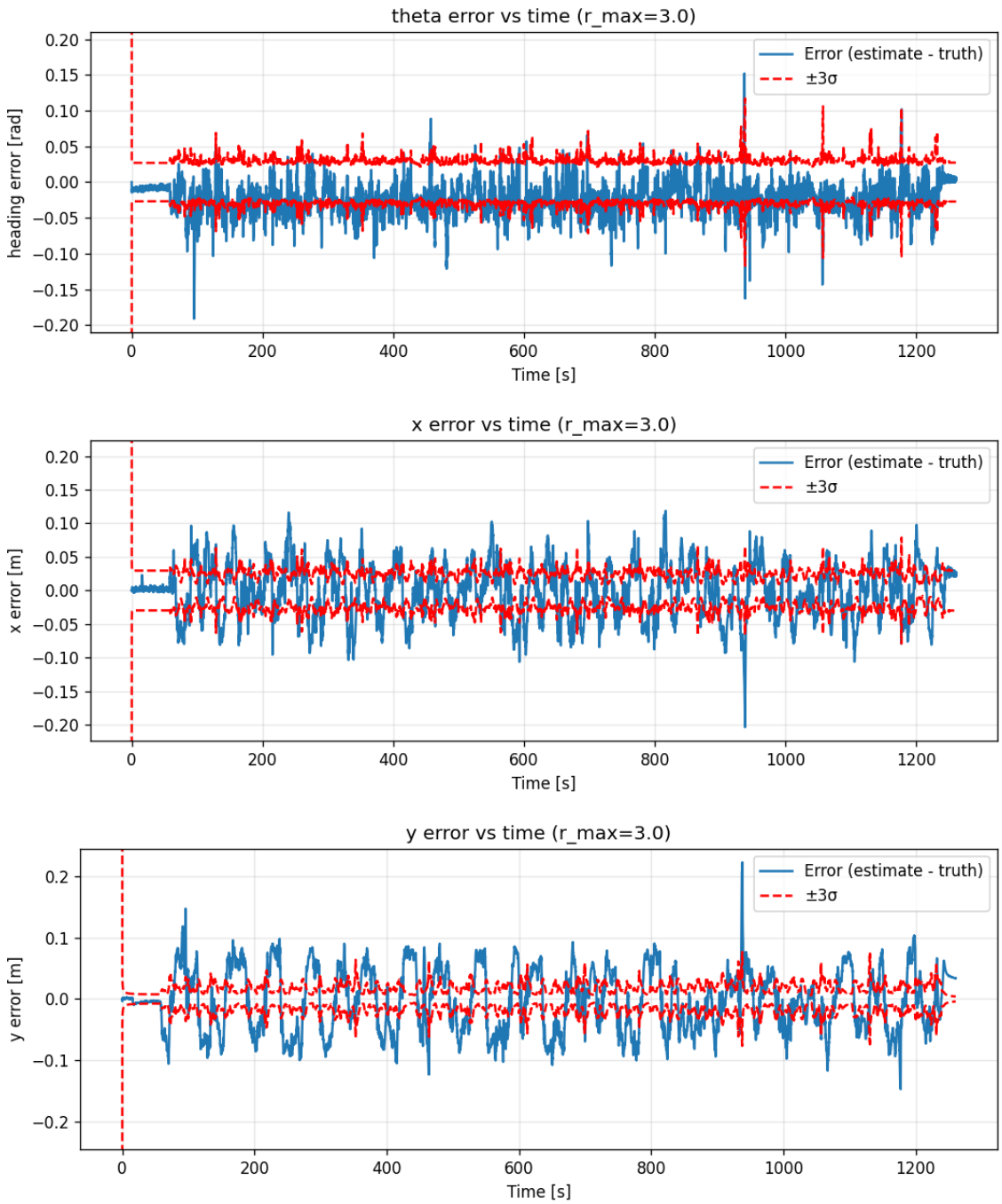


Figure8: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 3m$ with CRLB method

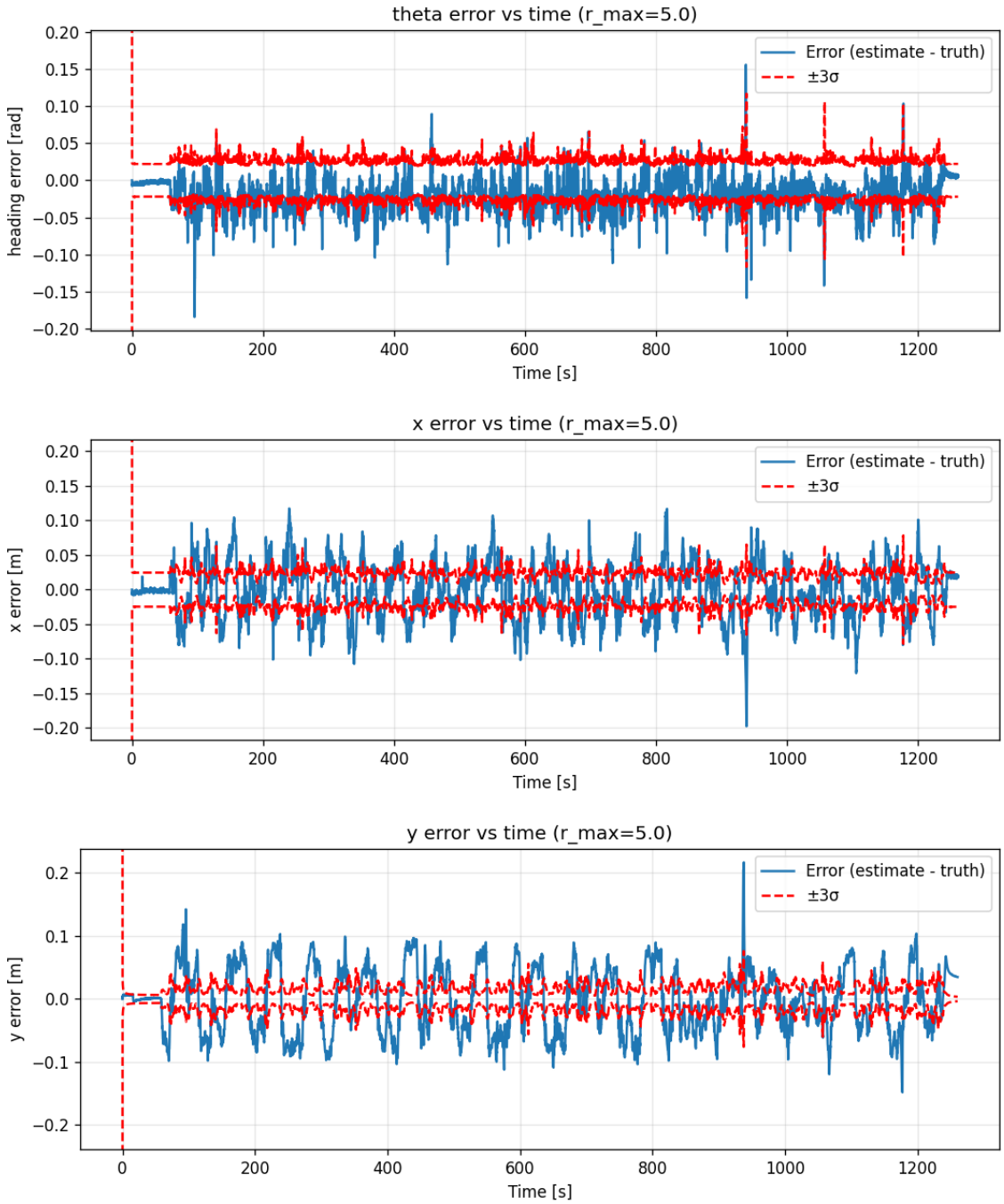


Figure9: Error of θ, x, y and $\pm 3\sigma$ bands plot for $r_{max} = 5m$ with CRLB method

Observation

Compared with question (a), question (c) shows **slightly narrower** bands since this method reduced linearization error in P_k. Nevertheless, the error still occasionally exits the $\pm 3\sigma$ band for same reason as previous questions.

Interpretation

In Q4(c), the Jacobians is computed using the true state. Since we only change where evaluate the Jacobian while everything else is identical. Innovation still uses $\hat{z} = h(\hat{x}^-)$, same gating, same Q/R, same time step. Because (a) already tracks closely and geometry is information-saturated, the resulting gain K and covariance P are nearly the same; you get only minor tightening (slightly more realistic $\pm 3\sigma$), not a dramatic change.

Optional note on Joseph update

We re-ran Q4 using the **Joseph form**

$$P_k = (I - K_k G_k) P_k^- (I - K_k G_k)^\top + K_k R_k K_k^\top.$$

Result: error curves and $\pm 3\sigma$ bands are essentially the same as the non-Joseph runs. This is expected here because the problem is well conditioned, and information is saturated for $r_{\max} \geq 3$.

Question 5 — Animation of EKF path

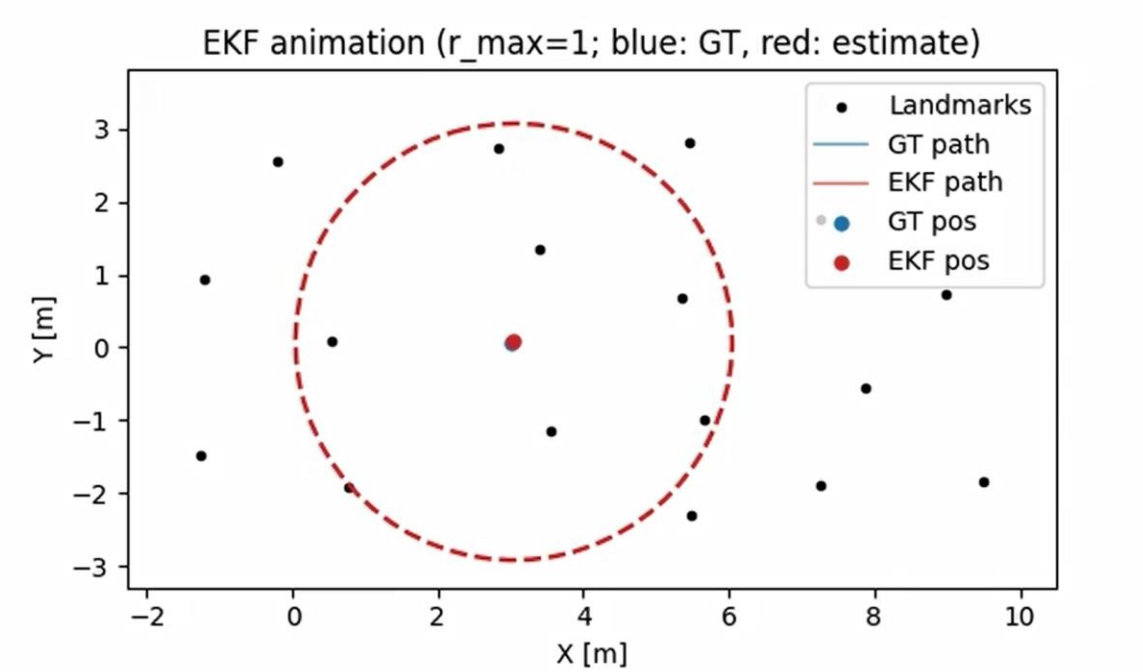


Figure10: Path of Ground Truth and Estimation at start

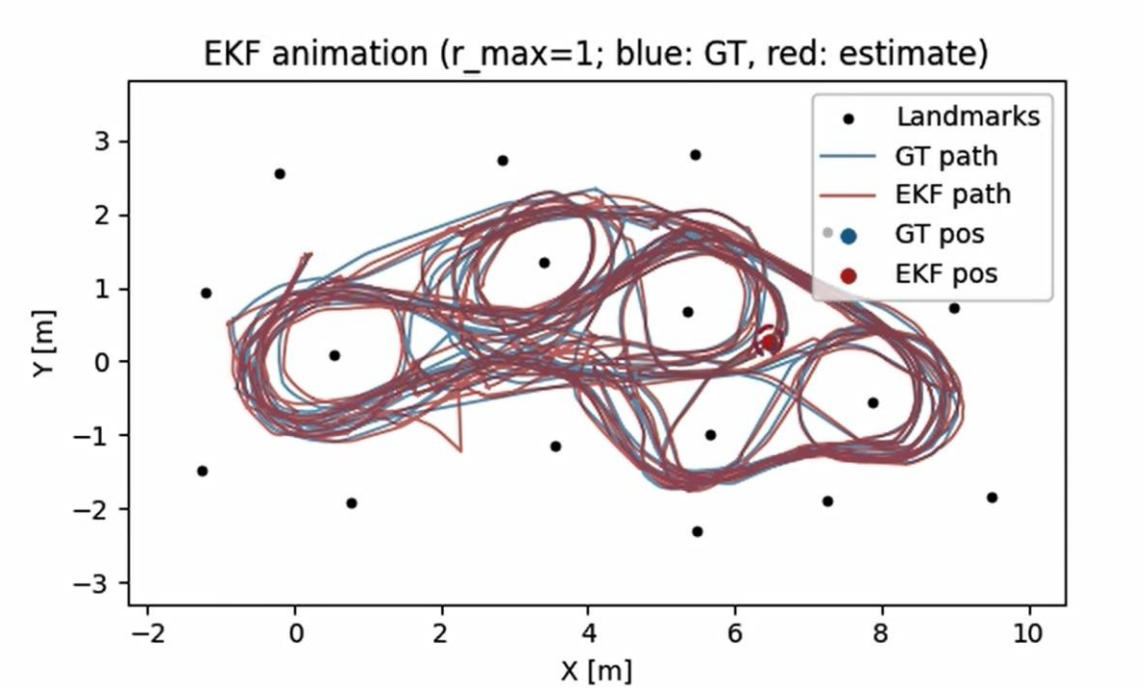


Figure11: Path of Ground Truth and Estimation during process

Observation

- **Start ($k = 0$)**. The estimate exactly overlaps the true robot position because we initialize with $\hat{x}_0 = x_0$. The red 3σ ellipse appears as a large circle, showing high initial uncertainty in (x, y).

- **While the true robot is still**. The true (blue) point stays fixed, but the estimate (red) may drift slowly toward the center of the map. The ellipse **does not shrink** yet. Reason: with $r_{\max} = 1$ there are few usable measurements, so the filter is mostly predicted with process noise and has little new information to tighten $\hat{P}_k$.

- **As soon as the robot starts moving**. The estimate snaps toward the true point and the ellipse shrinks quickly. Reason: motion plus nearby visible landmarks produce informative range–bearing updates; the Kalman gain becomes large and reduces the covariance.

- **After that (tracking phase)**. The estimate tracks the true path most of the time.

    - When geometry is good (several nearby landmarks), the tracking looks very accurate, and the ellipse stays small.

    - When geometry or visibility worsens (fewer landmarks, or a turn), the error grows a little bit and the ellipse expands slightly. Once informative measurements return, the ellipse tightens again.

Appendix

Assigment2.py

```
# AER1513_Assignment2_Feiyang

import os
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
from matplotlib.animation import FuncAnimation
import scipy.io

DATASET_PATH = r"C:\Users\89286\Downloads\dataset2.mat"

SAVE_PNG_DPI = 120
ANIM_INTERVAL_MS = 50
R_MAX_LIST = [5.0, 3.0, 1.0]

T_CLIP_SEC_FOR_YLIM = 60.0
YLIM_PERCENTILE = 100

RUN_Q4A = True
RUN_Q4B_POOR = True
RUN_Q4C_CRLB = True
RUN_Q5_ANIM = True

def wrap_angle(a: np.ndarray) -> np.ndarray:
    return (a + np.pi) % (2.0 * np.pi) - np.pi


def motion_model(x_prev: np.ndarray, u: np.ndarray, T: float) ->
np.ndarray:

    x, y, th = x_prev
    v_k, om_k = u
    x_n = x + T * v_k * np.cos(th)
    y_n = y + T * v_k * np.sin(th)
    th_n = wrap_angle(th + T * om_k)
    return np.array([x_n, y_n, th_n], dtype=float)
```

```python
def motion_jacobian_F(x_prev: np.ndarray, u: np.ndarray, T: float) ->
np.ndarray:

    th = x_prev[2]
    v_k = u[0]
    F = np.array([
        [1.0, 0.0, -T * v_k * np.sin(th)],
        [0.0, 1.0,  T * v_k * np.cos(th)],
        [0.0, 0.0,  1.0]
    ], dtype=float)
    return F

def process_noise_jacobian_G(x_prev: np.ndarray, T: float) ->
np.ndarray:

    th = x_prev[2]
    G = np.array([
        [T * np.cos(th), 0.0],
        [T * np.sin(th), 0.0],
        [0.0,            T  ]
    ], dtype=float)
    return G

def observation_model(x: np.ndarray, landmark_xy: np.ndarray, d: float)
-> np.ndarray:

    dx = landmark_xy[0] - (x[0] + d * np.cos(x[2]))
    dy = landmark_xy[1] - (x[1] + d * np.sin(x[2]))
    q = dx * dx + dy * dy
    r_pred = np.sqrt(max(q, 1e-12))
    b_pred = wrap_angle(np.arctan2(dy, dx) - x[2])
    return np.array([r_pred, b_pred], dtype=float)

def observation_jacobian_H(x: np.ndarray, landmark_xy: np.ndarray, d:
float) -> np.ndarray:

    dx = landmark_xy[0] - (x[0] + d * np.cos(x[2]))
    dy = landmark_xy[1] - (x[1] + d * np.sin(x[2]))
    q = dx * dx + dy * dy
    r = np.sqrt(max(q, 1e-12))
    inv_q = 1.0 / max(q, 1e-12)
```

```
    H = np.zeros((2, 3), dtype=float)
    H[0, 0] = -dx / r
    H[0, 1] = -dy / r
    H[1, 0] =  dy * inv_q
    H[1, 1] = -dx * inv_q
    H[0, 2] = (dx * d * np.sin(x[2]) - dy * d * np.cos(x[2])) / r
    H[1, 2] = -1.0 - ((dx * d * np.cos(x[2]) + dy * d * np.sin(x[2])) *
inv_q)
    return H

if not os.path.exists(DATASET_PATH):
    raise FileNotFoundError(f"dataset2.mat not found at:
{DATASET_PATH}")

data = scipy.io.loadmat(DATASET_PATH)

t = data["t"].ravel().astype(float)
r = np.array(data["r"], dtype=float)          # (N, M) ranges
b = np.array(data["b"], dtype=float)          # (N, M) bearings
l = np.array(data["l"], dtype=float)          # landmarks: (M, 2) or (2,
M)
if l.shape[0] == 2 and l.shape[1] != 2:
    l = l.T  # ensure (M, 2)

d = float(np.array(data["d"]).squeeze())
v = data["v"].ravel().astype(float)           # (N,)
om = data["om"].ravel().astype(float)         # (N,)
r_var = float(np.array(data["r_var"]).squeeze())
b_var = float(np.array(data["b_var"]).squeeze())
v_var = float(np.array(data["v_var"]).squeeze())
om_var = float(np.array(data["om_var"]).squeeze())

x_true = data["x_true"].ravel().astype(float)
y_true = data["y_true"].ravel().astype(float)
th_true = data["th_true"].ravel().astype(float)

N = len(t)
M = l.shape[0]
T = 0.1
```

```
def run_ekf(r_max_list, x0, P0, linearize_at="estimate", suffix="",
save_prefix=""):

    Q_control = np.array([[v_var, 0.0],
                          [0.0,   om_var]], dtype=float)
    R_single = np.array([[r_var, 0.0],
                         [0.0,   b_var]], dtype=float)

    results = {}
    for r_max in r_max_list:
        x_est = np.zeros((N, 3), dtype=float)
        P_est = np.zeros((N, 3, 3), dtype=float)
        x_est[0, :] = x0
        P_est[0, :, :] = P0

        for k in range(1, N):
            # Prediction
            u_km1 = np.array([v[k - 1], om[k - 1]], dtype=float)
            x_prev = x_est[k - 1, :]
            P_prev = P_est[k - 1, :, :]

            # propagate mean at estimated previous state
            x_pred = motion_model(x_prev, u_km1, T)

            # Jacobians for covariance propagation
            if linearize_at == "truth":
                x_lin_km1 = np.array([x_true[k - 1], y_true[k - 1],
th_true[k - 1]])
                F_km1 = motion_jacobian_F(x_lin_km1, u_km1, T)
                G_km1 = process_noise_jacobian_G(x_lin_km1, T)
            else:
                F_km1 = motion_jacobian_F(x_prev, u_km1, T)
                G_km1 = process_noise_jacobian_G(x_prev, T)

            Q_k = G_km1 @ Q_control @ G_km1.T
            P_pred = F_km1 @ P_prev @ F_km1.T + Q_k

            y_blocks = []
            y_hat_blocks = []
            H_blocks = []
```

```
        r_k = r[k, :]
        b_k = b[k, :]

        for j in range(M):
            r_lk = r_k[j]
            b_lk = b_k[j]
            if r_lk > 0.0 and r_lk < r_max:
                lm = l[j, :]
                y_meas = np.array([r_lk, b_lk], dtype=float)

                # predicted measurement ALWAYS at x_pred
                y_hat = observation_model(x_pred, lm, d)

                if linearize_at == "truth":
                    x_lin_k = np.array([x_true[k], y_true[k],
th_true[k]])
                    H_k = observation_jacobian_H(x_lin_k, lm, d)
                else:
                    H_k = observation_jacobian_H(x_pred, lm, d)

                y_blocks.append(y_meas)
                y_hat_blocks.append(y_hat)
                H_blocks.append(H_k)

        if len(y_blocks) > 0:
            Y = np.vstack(y_blocks)           # (m, 2)
            Y_hat = np.vstack(y_hat_blocks)   # (m, 2)
            H = np.vstack(H_blocks)           # (2m, 3)
            m = Y.shape[0]
            R_block = np.kron(np.eye(m), R_single)

            residual = Y - Y_hat
            # wrap bearing residuals
            residual[:, 1] = wrap_angle(residual[:, 1])
            res_vec = residual.reshape(-1)

            S = H @ P_pred @ H.T + R_block
            K = P_pred @ H.T @ np.linalg.inv(S)

            x_upd = x_pred + K @ res_vec
            x_upd[2] = wrap_angle(x_upd[2])
```

```
            P_upd = (np.eye(3) - K @ H) @ P_pred
        else:
            x_upd, P_upd = x_pred, P_pred

        x_est[k, :] = x_upd
        P_est[k, :, :] = P_upd

    results[r_max] = (x_est, P_est)

    err_x = x_est[:, 0] - x_true
    err_y = x_est[:, 1] - y_true
    err_th = wrap_angle(x_est[:, 2] - th_true)

    sig_x = 3.0 * np.sqrt(np.maximum(P_est[:, 0, 0], 0.0))
    sig_y = 3.0 * np.sqrt(np.maximum(P_est[:, 1, 1], 0.0))
    sig_th = 3.0 * np.sqrt(np.maximum(P_est[:, 2, 2], 0.0))

    def save_err(name, e, s, ylab):

        if T_CLIP_SEC_FOR_YLIM > 0.0:
            idx = t >= T_CLIP_SEC_FOR_YLIM
        else:
            idx = np.ones_like(t, dtype=bool)

        e_use = e[idx] if np.any(idx) else e
        s_use = s[idx] if np.any(idx) else s

        p_e = float(np.percentile(np.abs(e_use), YLIM_PERCENTILE))
if e_use.size else 0.0
        p_s = float(np.percentile(s_use,          YLIM_PERCENTILE))
if s_use.size else 0.0
        y_lim = 1.10 * max(p_e, p_s, 1e-6)  # small margin

        plt.figure(figsize=(9, 3.6))
        plt.plot(t, e, label='Error (estimate - truth)')
        plt.plot(t,  s, 'r--', label='±3σ')
        plt.plot(t, -s, 'r--')
        plt.ylim(-y_lim, y_lim)

        plt.xlabel('Time [s]')
        plt.ylabel(ylab)
```

```
            plt.title(f'{name} error vs time (r_max={r_max})')
            plt.grid(True, alpha=0.3)
            plt.legend()
            plt.tight_layout()

            fn =
f'{save_prefix}{name}_error_rmax{int(r_max)}{suffix}.png'
            plt.savefig(fn, dpi=SAVE_PNG_DPI)
            plt.close()

        save_err('x',     err_x,  sig_x,  'x error [m]')
        save_err('y',     err_y,  sig_y,  'y error [m]')
        save_err('theta', err_th, sig_th, 'heading error [rad]')

    return results

# Q4(a)
x0_good = np.array([x_true[0], y_true[0], th_true[0]], dtype=float)
P0_good = np.diag([1.0, 1.0, 0.1])

results_good = {}
if RUN_Q4A:
    results_good = run_ekf(R_MAX_LIST, x0_good, P0_good,
                           linearize_at="estimate", suffix="",
save_prefix="")

# Q4(b)
if RUN_Q4B_POOR:
    x0_bad = np.array([1.0, 1.0, 0.1], dtype=float)  # per spec
    _ = run_ekf(R_MAX_LIST, x0_bad, P0_good,
                linearize_at="estimate", suffix="_badinit",
save_prefix="")

# Q4(c)
if RUN_Q4C_CRLB:
    _ = run_ekf(R_MAX_LIST, x0_good, P0_good,
                linearize_at="truth", suffix="_crlb", save_prefix="")

#Q5
if RUN_Q5_ANIM:
    if 1.0 not in (results_good or {}):
```

```
        results_good = run_ekf([1.0], x0_good, P0_good,
                               linearize_at="estimate", suffix="",
save_prefix="")
    x_est_anim, P_est_anim = results_good[1.0]

    fig, ax = plt.subplots()
    ax.set_aspect('equal', adjustable='box')

    # Landmarks: black, static
    ax.scatter(l[:, 0], l[:, 1], c='k', marker='.', label='Landmarks')

    # Optional paths
    line_true, = ax.plot([], [], '-', color='tab:blue', lw=1.2,
alpha=0.6, label='GT path')
    line_est,  = ax.plot([], [], '-', color='tab:red',  lw=1.2,
alpha=0.6, label='EKF path')

    # Moving dots (blue: truth, red: EKF)
    point_true, = ax.plot([], [], 'o', color='tab:blue', ms=5, label='GT
pos')
    point_est,  = ax.plot([], [], 'o', color='tab:red',  ms=5,
label='EKF pos')

    # 3-sigma covariance ellipse (red)
    ellipse = Ellipse(xy=(0, 0), width=0.0, height=0.0, angle=0.0,
                      edgecolor='tab:red', linestyle='--', fill=False,
lw=1.8)
    ax.add_patch(ellipse)

    # Axes limits
    all_x = np.concatenate([x_true, x_est_anim[:, 0], l[:, 0]])
    all_y = np.concatenate([y_true, x_est_anim[:, 1], l[:, 1]])
    margin = 1.0
    ax.set_xlim(all_x.min() - margin, all_x.max() + margin)
    ax.set_ylim(all_y.min() - margin, all_y.max() + margin)
    ax.set_xlabel('X [m]')
    ax.set_ylabel('Y [m]')
    ax.set_title('EKF animation (r_max=1; blue: GT, red: estimate)')
    ax.legend(loc='upper right')

    def update(frame: int):
```

```
        i = frame

        line_true.set_data(x_true[:i+1], y_true[:i+1])
        line_est.set_data(x_est_anim[:i+1, 0], x_est_anim[:i+1, 1])

        point_true.set_data([x_true[i]], [y_true[i]])
        point_est.set_data([x_est_anim[i, 0]], [x_est_anim[i, 1]])

        P_xy = P_est_anim[i, 0:2, 0:2]
        evals, evecs = np.linalg.eigh(P_xy)
        order = np.argsort(evals)[::-1]
        evals = np.maximum(evals[order], 0.0)
        evecs = evecs[:, order]
        width = 2.0 * 3.0 * np.sqrt(evals[0])
        height = 2.0 * 3.0 * np.sqrt(evals[1])
        angle_deg = np.degrees(np.arctan2(evecs[1, 0], evecs[0, 0]))
        ellipse.set_center((x_est_anim[i, 0], x_est_anim[i, 1]))
        ellipse.width = width
        ellipse.height = height
        ellipse.angle = angle_deg
        return line_true, line_est, point_true, point_est, ellipse

    anim = FuncAnimation(fig, update, frames=N,
interval=ANIM_INTERVAL_MS, blit=True)

    fps_val = max(1, int(round(1.0 / T))) if T > 0 else 10
    anim.save('ekf_path_animation.mp4', writer='ffmpeg', fps=fps_val)
    print('Saved MP4: ekf_path_animation.mp4')
    plt.close(fig)
```